```
(1..100).sum  # => 5050
```

Addition only assumes the elements respond to +:

```
[[1, 2], [2, 3], [3, 4]].sum    # => [1, 2, 2, 3, 3, 4]
%w(foo bar baz).sum             # => "foobarbaz"
{a: 1, b: 2, c: 3}.sum # => [:b, 2, :c, 3, :a, 1]
```

The sum of an empty collection is zero by default, but this is customizable:

```
[].sum    # => 0
[].sum(1) # => 1
```

If a block is given, `sum` becomes an iterator that yields the elements of the collection and sums the returned values:

```
(1..5).sum {|n| n * 2 } # => 30
[2, 4, 6, 8, 10].sum    # => 30
```

The sum of an empty receiver can be customized in this form as well:

```
[].sum(1) {|n| n**3} # => 1
```

Defined in `active_support/core_ext/enumerable.rb`.

9.2 `index_by`

The method `index_by` generates a hash with the elements of an enumerable indexed by some key.

It iterates through the collection and passes each element to a block. The element will be keyed by the value returned by the block:

```
invoices.index_by(&:number)
# => {'2009-032' => <Invoice ...>, '2009-008' => <Invoice ...>, ...}
```

Keys should normally be unique. If the block returns the same value for different elements no collection is built for that key. The last item will win.

Defined in `active_support/core_ext/enumerable.rb`.

9.3 `many?`

The method `many?` is shorthand for `collection.size > 1`:

```
<% if pages.many? %>
  <%= pagination_links %>
<% end %>
```

If an optional block is given, `many?` only takes into account those elements that return true:

```
@see_more = videos.many? {|video| video.category == params[:category]}
```

Defined in `active_support/core_ext/enumerable.rb`.

9.4 `exclude?`

The predicate `exclude?` tests whether a given object does **not** belong to the collection. It is the negation of the built-in `include?`:

```
to_visit << node if visited.exclude?(node)
```

Defined in `active_support/core_ext/enumerable.rb`.

10 Extensions to `Array`

10.1 Accessing

Active Support augments the API of arrays to ease certain ways of accessing them. For example, `to`returns the subarray of elements up to the one at the passed index:

```
%w(a b c d).to(2) # => %w(a b c)
[].to(7)          # => []
```